VitalFlow — Security & RBAC Architecture

> Multi-practice, multi-tenant healthcare SaaS. Hard isolation between practices, cast-iron separation between staff and patient users, auditable clinical actions, and controlled platform support access. Postgres RLS is the real boundary; every other layer is defense in depth.

1. Tenancy model

Tenant = practice. A practice is the unit of:

- Data isolation (RLS scope)
- Billing (one subscription per practice)
- HIPAA BAA (one agreement per practice)
- Compliance audit (one audit log per practice)
- Branding / domain (one subdomain per practice, optional)

A practice can have multiple **locations** (already modeled in locations). Future: an **organization** entity can own multiple practices (parent/child). Not built today — `tenants` is the top object.

Every tenant-scoped table already carries `tenant_id uuid NOT NULL`. RLS policies gate via `public.current_user_tenant_ids()` + `public.has_permission()`.

2. User model

Four disjoint user "kinds." Enforced by `profiles.user_kind` + table-level check constraints:

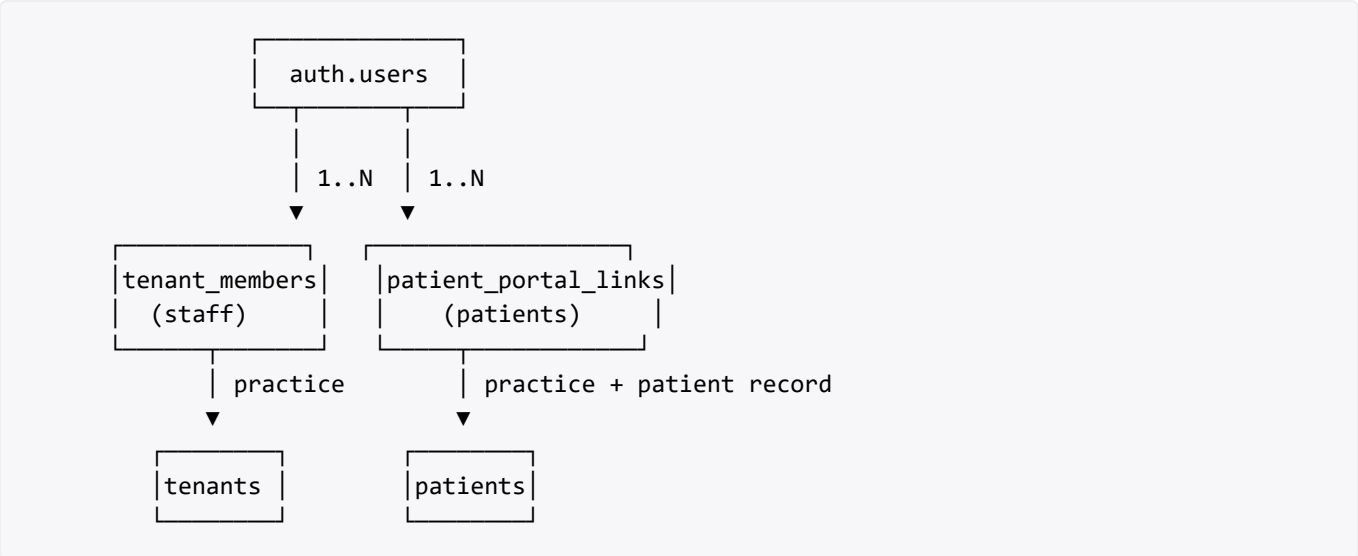
Kind	Stored where	Auth source	Tenant scope
`staff`	`auth.users` + `profiles`	Password / SSO	1..N practices via `tenant_members`
`patient`	`auth.users` + `profiles` (distinct)	Password / magic link	1..N practices via `patient_portal_links` (never via `tenant_members`)
`platform`	`auth.users` + `profiles` + `platform_admins`	Password + WebAuthn (required)	All practices (read-only by default; impersonation required for writes)
`service`	no human auth — token bound to server	Service-account JWT	Platform-wide, restricted by audit scope

Hard separation rules (enforced by DB):

- A single `auth.users` row is exactly one kind. `profiles.user_kind` is `NOT NULL`, check-constrained to `('staff','patient','platform','service')`, and immutable after creation.
- `tenant_members` can only reference profiles with `user_kind = 'staff'`. Trigger enforces this at insert/update.
- `patient_portal_links` can only reference profiles with `user_kind = 'patient'`. Trigger enforces this.
- Staff can never log in to the patient portal; patients can never access the staff workspace. Both surfaces check `user_kind` before rendering.

Why not one table with a `is_staff` flag? Healthcare support workflows need to create patient records for staff members who are also patients at their own practice. Solution: **two distinct `auth.users` accounts with different email aliases.** Same human, different digital identities — because a user logged in as "Dr. Smith" must not silently see his own medical record through the staff console.

3. Practice membership model



`tenant_members` **(staff-only, multi-role, multi-tenant)** — one row per `(tenant_id, user_id)`. Roles are `staff_role[]` — a staff person can be both `physician` and `office_admin` at one practice, or `nurse_ma` at practice A and `scheduler` at practice B.

`patient_portal_links` **(patient-only, links auth user to patient record)** — one row per `(tenant_id, patient_id)`. A patient seen at three practices gets three rows. Each link is verified (`verified_at`) before portal access is granted.

`platform_admins` **(global, no tenant)** — out-of-band table for super_admins. Not referenced by tenant RLS — platform admins access data only via impersonation or signed RPCs.

4. Role model

Three role namespaces, zero overlap.

Staff roles (`public.staff_role` enum, stored in `tenant_members.roles[]`)

Role	Scope
`practice_owner`	Full control of the practice. Only owner can delete the practice.
`office_admin`	User management, settings, billing config. No clinical sign, no Rx.
`physician`	Full clinical rights including signing notes and creating/signing prescriptions.
`nurse_ma`	Clinical read + write, no sign, no Rx creation. Can prep charts + take vitals.
`scheduler`	Scheduling + patient demographics only. No clinical content.
`biller`	Billing + coding. Read-only clinical access for coding. No write to charts.

A staff user can carry multiple roles in one membership (`roles: ['physician', 'practice_owner']`). Permissions are the **union**.

Platform roles (`public.platform_role` enum, stored in `platform_admins.role`)

Role	Purpose
`super_admin`	Tenant lifecycle, billing escalations, incident response, impersonation.

Future platform roles (support_l1, compliance_officer) slot in here.

Patient

Not a "role" — it's a user **kind**. Patients don't have a role value in any enum; their capability set is fixed and bound to their own patient record via `patient_portal_links` .

5. Permission strategy

Permissions are granular strings — `<domain>:<action>` . Roles grant permissions; application code checks permissions, never roles directly. This keeps role definitions flexible without rewriting every RLS policy.

Full permission catalog

Domain	Permissions
Clinical	`clinical:read` , `clinical:write` , `clinical:sign` , `clinical:amend`

Patient	`patient:read`, `patient:write`, `patient:demographics_only`
Rx	`rx:create`, `rx:sign`, `rx:refill`
Orders	`order:create`, `order:resolve`
Schedule	`schedule:read`, `schedule:write`
Billing	`billing:read`, `billing:write`, `billing:collect`, `billing:adjust`, `billing:write_off`
Admin	`admin:tenant`, `admin:users`, `admin:billing_config`, `admin:integrations`
Audit	`audit:read`
AI	`ai:invoke`, `ai:train`
Patient self	`self:read`, `self:write`, `self:message_care_team`, `self:book_appointment`

Role → permission map

(Complete mapping in `packages/auth/src/rbac.ts` and in the Postgres `has_permission()` function — the two must stay in sync.)

Role / Kind	Gets
`practice_owner`	Everything in its practice (all clinical, billing, admin, audit). Cannot grant super_admin.
`office_admin`	`admin:users`, `admin:tenant` (limited), `admin:billing_config`, `admin:integrations`, all `billing:*`, `schedule:*`, `patient:read`, `patient:write`, `audit:read`.
`physician`	All `clinical:*`, all `rx:*`, all `order:*`, `patient:read`, `patient:write`, `schedule:read`, `ai:invoke`.
`nurse_ma`	`clinical:read`, `clinical:write` (NOT `clinical:sign`), `order:create` (NOT `rx:*`), `patient:read`, `patient:write`, `schedule:read`, `ai:invoke`.
`scheduler`	`schedule:*`, `patient:read` + `patient:demographics_only` (write restricted to demographics via DB trigger).
`biller`	`billing:*`, `clinical:read` (read-only for coding), `patient:read`.
Patient (kind)	Fixed: `self:*`. No access to others' records. Enforced via RLS join on `patient_portal_links`.
`super_admin`	**None by default.** Must impersonate a staff user to access any practice data; every action audited.

Enforcement layers (in order of authority):

1. **Postgres RLS** — the only one that actually protects data. Every tenant-scoped table has `FOR SELECT / INSERT / UPDATE / DELETE` policies that call `has_permission(perm, tenant_id)`.
2. **Server Action / Route Handler guards** — `requirePermission('clinical:sign')` at the top of every write endpoint. Short-circuits before touching the DB.
3. **UI guards** — `AuthGuard` hides UI a user can't use. UX only; never the security boundary.

6. Middleware & authorization enforcement

Next.js middleware (`packages/auth/src/middleware.ts`)

Runs on every request. Responsibilities:

1. **Refresh Supabase session cookie** (idempotent, already implemented).
2. **Resolve active tenant.** Derivation priority: `x-vitalflow-tenant` header (SSR fetches) → `vf_tenant` cookie (post-login pick) → subdomain (multi-tenant prod) → first tenant in user's membership list.
3. **Block unauthenticated access** to `/(app)/*` — redirect to `/login` with `?next=` pinned.
4. **Inject tenant context** into downstream requests via a signed `x-vitalflow-tenant` request header.
5. **Check impersonation state.** If JWT carries `impersonating: true` with expired `imp_exp`, revoke and redirect to `/support`.

Server-side guards (`packages/auth/src/guards.ts`)

Used at the top of Server Components and Server Actions:

```
import { requirePermission, requireSession, requireSurface } from "@vitalflow/auth/guards";

export default async function PatientsPage() {
  const session = await requireSession();
  requirePermission(session, "patient:read");
  requireSurface(session, "provider");
  // ... safe to render
}
```

`requirePermission` throws a `VitalFlowError('FORBIDDEN')` that the Next error boundary renders as a 403 page. These guards are defense in depth; the underlying query still goes through RLS.

Route-group layouts

The `(app)/admin/layout.tsx` and `(app)/my/layout.tsx` already gate their surfaces by role. Extend for permission-level checks as features land.

7. Support impersonation rules

Platform support is the riskiest vector. Every rule below is enforced in code + policy.

1. **Only `super_admin` can impersonate.** Platform role check happens in the `impersonate_user()` RPC.

2. **Target must exist and be staff.** Patient impersonation is **prohibited** — it creates HIPAA exposure with no support benefit. Staff only.
3. **Reason is mandatory.** Free-text field, min 20 chars. Logged to audit.audit_events with every mutation.
4. **Time-boxed.** Default 60 minutes, max 4 hours. Expiry auto-revokes; expired tokens can't mutate.
5. **Four-eyes (production).** A second `super_admin` approves impersonation sessions in production. Dev/staging allow single-admin with a different reason prefix (`[ENG]`).
6. **Visible banner.** Client shows `IMPERSONATING <user> · ends in <N>min` fixed at the top; dismissible only by ending the session.
7. **Write restrictions during impersonation.**
   - **Blocked:** signing notes, signing prescriptions, approving claims, billing adjustments, user-management changes.
   - **Allowed:** read, create draft notes, post diagnostic/support data, reassign tasks.
   - Enforced by `has_permission()` → reads impersonation context from session and strips the `:sign` , `:adjust` , `:write_off` permissions.
8. **Audited.** Every row written during impersonation carries the `impersonator_id` in the audit entry. Monthly audit review by compliance.

Data shape: `impersonation_sessions` table + JWT claims (`impersonator_id` , `imp_exp`). RPCs `impersonate_start` , `impersonate_end` write and revoke the session row; row existence is authoritative.

8. Invitation flow

Staff invite

```
office_admin/practice_owner           invitee
        |                                |
        |── POST /api/invitations ──────▶ | (email sent)
        |                                |
        |                                |── click link (token in URL)
        |                                |
        |                                |── /accept-invite → sign in / sign up (Supabase auth)
        |                                |
        |                                |── POST /api/invitations/accept (token)
        |                                |
        |◀─── member added to tenant ────| (trigger creates tenant_members row)
```

- `invitations` table already exists with `token_hash` , `expires_at` (7d default), `status` .
- Token is signed + hashed; only the hash is stored server-side. Email carries the raw token.
- Invites are `UNIQUE (tenant_id, email, status)` — can't double-invite.
- Revocation: set `status = 'revoked'` ; token becomes unusable.

- Accepting requires `auth.uid()` matches an auth.users row whose email equals `invitations.email` (case-insensitive).
- Audit: invite created, viewed, accepted, revoked — all logged.

Patient portal invite

Different flow — patient record exists first, then invite to claim the portal:

1. Staff creates patient record (MRN, demographics).
2. Patient requests portal access OR staff clicks "Invite to portal" → creates a `patient_portal_invites` row (table to add) with a tokenized link.
3. Patient clicks, signs up with Supabase (email + password or magic link).
4. Patient verifies identity via DOB + MRN or known-good phone code.
5. System creates `patient_portal_links` row linking `auth.users.id` ↔ `patients.id`.
6. Patient can now access `/my/*` surfaces scoped to their own record.

The verification step is what stops a hostile patient from linking to someone else's chart.

SSO provisioning (future)

When a practice enables SSO:

- `sso_configurations` row with IdP metadata, role claim name, role mapping
- On first IdP login, middleware reads asserted claims, maps to `staff_role[]`, inserts `tenant_members` — **only if** the tenant has JIT provisioning enabled.
- Default-deny if role claim missing: user gets an error, admin must fix mapping or invite explicitly.

9. Offboarding flow

Revoking access has to be seconds-fast and complete.

Staff offboarding

Triggered by `DELETE /api/members/{user_id}` or admin UI "remove member":

1. **Soft-delete the tenant_members row** (`deleted_at = now()`).
2. **Sign user out everywhere** via Supabase `auth.signOut({ scope: 'global' })` on all their active sessions. RLS picks up immediately because `current_user_tenant_ids()` no longer returns that tenant.
3. **Reassign pending work.** A background job (future) walks tasks + unsigned notes + open orders assigned to the user and:
    - Reassigns to their manager (`reports_to` — future column) or to the practice default queue

    - Never overwrites `created_by` / `ordering_provider_id` on historical records — those are immutable for compliance
4. **Revoke active invitations** that user issued (`status = 'revoked'`).
5. **Rotate any credentials** the user created (API keys, webhook secrets) via an operations task.
6. **Audit log** — offboarding reason, actor, affected user, timestamp.

If the user belongs to **multiple practices**, only the one practice's membership is removed; other memberships stay active.

Patient offboarding

Rare (typically on death, account merge, or HIPAA request to close portal access). Flow:

1. Mark `patient_portal_links` row as revoked (`deleted_at`).
2. Sign user out globally.
3. **Do NOT delete the `patients` record** — HIPAA requires retention. Only the portal access is removed.
4. For HIPAA "right to erasure" requests: separate compliance workflow, not part of standard offboarding, handled via ticketed RPC by `super_admin` + compliance approval.

Platform admin offboarding

Immediate revocation:

1. Delete `platform_admins` row.
2. Invalidate any active `impersonation_sessions` rows (`revoked_at = now()`).
3. Revoke WebAuthn credentials, API keys.
4. Audit entry with reason + approving officer.

10. Security risks & mitigations

#	Risk	Mitigation
1	**Tenant confusion** — user A's token reads tenant B's data	Every tenant-scoped table has RLS keyed on `current_user_tenant_ids()` . Periodic fuzz tests.
2	**Staff ↔ patient confusion** — staff token accesses patient portal	`user_kind` discriminator + table-level triggers on `tenant_members` and `patient_portal_links` . `/my/*` layout rejects `user_kind != 'patient'` .
3	**Privilege escalation** — user promotes self to `practice_owner`	Role changes require `admin:users` permission; trigger forbids granting `practice_owner` except by an existing `practice_owner` or `super_admin` . All role changes audited.
4	**Impersonation abuse**	Four-eyes in prod, time-box, write restrictions, banner, monthly audit review.

5	**Insider threat** — staff reads charts they shouldn't	PHI reads logged to audit (sampled by default; tagged on "high-sensitivity" records). Anomaly detection (future).
6	**Token theft**	Short TTL (1h default), `httpOnly + SameSite=Lax + Secure`, rotation on role/tenant change, WebAuthn for super_admin.
7	**SSO misconfiguration**	Default-deny on missing role claim; `tenant_members` insert requires explicit mapping. Admin alerted on unmapped claims.
8	**Session fixation / replay**	Rotate session on role/tenant change; store session binding (IP or device fingerprint, configurable).
9	**BAA gap** — new tenant before contract signed	Trigger blocks PHI writes (patients/encounters/etc.) until `tenants.hipaa_baa_signed = true`. Preview env has a constraint that BAA must be false.
10	**Audit tampering**	`audit.audit_events`: `REVOKE UPDATE, DELETE` from all roles except `postgres`; daily hash-chain export (future) to WORM storage.
11	**Patient-portal account takeover**	Email + DOB + MRN challenge OR phone OTP for portal activation. Post-link verification via `verified_at` timestamp.
12	**Orphaned memberships** (user deleted in auth.users)	`ON DELETE CASCADE` from `auth.users` through `tenant_members`, `patient_portal_links`, `platform_admins`, `impersonation_sessions`.
13	**Service-account sprawl**	Every service token has an owning human + expiration. Rotate quarterly. Tracked in `integration_connections`.
14	**RLS bypass via SECURITY DEFINER drift**	Supabase advisors run in CI + weekly; PRs that add SECURITY DEFINER functions require `@vitalflow/security` approval.

Artifacts

- **Migration:** supabase/migrations/20260416000014_rbac_redesign.sql
- **Types:** packages/types/src/tenancy/index.ts — `StaffRole`, `PlatformRole`, `Permission`, `UserKind`
- **RBAC module:** packages/auth/src/rbac.ts — permission catalog + role map (mirrors the Postgres function)
- **Guards:** packages/auth/src/guards.ts — `requireSession`, `requirePermission`, `requireSurface`, `requireImpersonationContext`
- **Impersonation helpers:** packages/auth/src/impersonation.ts